VORTEX: A LIGHTWEIGHT CROSS-PLATFORM GPU-ACCELERATED ENGINE FOR REAL-TIME OBJECT TRACKING AND RENDERING

A PREPRINT

Prakharkumar Srivastava[**]
Flam, Bangalore, India
IIT Roorkee, India

Nikhil Karthik[*]
Flam, Bangalore, India
IIIT Dharwad, India

Aman Agrawal[*]
Flam, Bangalore, India

Aloori Raj Aryan
Flam, Bangalore, India
IIT BHU, Varanasi, India

April 15, 2026

ABSTRACT

Real-time 6-DoF pose estimation on mobile and embedded platforms is constrained by both computational limitations and geometric robustness requirements. Dense SLAM pipelines and learned pose regression models typically require substantial compute and memory resources, making them difficult to deploy efficiently on edge devices. Classical feature-based tracking offers a lightweight alternative, but most existing implementations rely on purely algebraic point-matching formulations—such as homography estimation—that lack explicit 3D geometric constraints and often become unstable under viewpoint changes, scale variation, or partial visibility.

In this work, we present VORTEX, a hardware-aware and geometry-aware tracking framework designed specifically for edge devices. We first redesign the foundational algebraic tracking stages through two optimized implementations: a CPU pipeline leveraging SIMD vectorization and a GPU-resident feature pipeline using custom fragment and compute shaders for keypoint detection, descriptor extraction, and parallel descriptor matching. Building on this accelerated feature backbone, we introduce a render-and-track formulation on the SE(3) manifold that incorporates explicit object geometry through synthetic reference rendering and direct 2D–3D correspondence lifting.

To further improve optimization stability, we propose a physics-inspired formulation for SE(3) pose estimation. Instead of directly minimizing reprojection error, each 2D–3D correspondence is modeled as a virtual spring, where the smallest residual defines a natural length and adaptive tension forces guide the optimization. This results in a weighted Gauss–Newton update that emphasizes geometrically consistent correspondences while suppressing outliers and local degeneracies. The formulation provides improved convergence behavior and robustness under partial visibility and non-uniform feature distributions.

Experiments on Android-based mobile hardware demonstrate that while GPU-accelerated algebraic tracking improves computational throughput, the proposed geometry-aware SE(3) refinement with tension-based optimization significantly enhances pose stability and robustness for real-time 6-DoF object tracking without persistent mapping.

Keywords 6-DoF pose estimation · Edge-aware vision systems · Hardware-aware computer vision · GPU-accelerated feature tracking · Render-and-track methods · SE(3) pose optimization · Mobile augmented reality

1 Introduction

Accurate six degrees-of-freedom (6-DoF) pose estimation is fundamental to augmented reality (AR), where virtual content must remain geometrically aligned with physical objects under strict real-time constraints. Although modern

[*]Equal contribution.

visual SLAM systems and learned pose regression models achieve high accuracy, they typically rely on dense mapping, global optimization, or large-scale training. Such approaches introduce significant computational and memory overhead, making them difficult to deploy on mobile and embedded platforms where latency, power, and resource constraints are critical.

Classical feature-based tracking pipelines provide a computationally efficient alternative and remain widely used in mobile vision systems. These pipelines typically consist of image preprocessing, keypoint detection, descriptor extraction, and feature matching, followed by motion estimation using algebraic formulations such as homography estimation or optical flow. While lightweight, two major challenges remain. First, feature extraction and matching often dominate the runtime on resource-constrained devices, creating a critical computational bottleneck. Second, purely algebraic formulations lack explicit 3D geometric reasoning, making them susceptible to instability under out-of-plane rotations, rapid scale changes, and planar degeneracies.

In this work, we investigate how hardware-aware acceleration and geometric reasoning can be jointly leveraged to enable efficient and stable tracking on edge devices. We develop a three-stage tracking framework that progressively addresses both computational and geometric limitations. First, we implement a highly optimized CPU-based algebraic tracking pipeline that exploits SIMD vectorization (e.g., ARM NEON) to accelerate feature detection, descriptor computation, and matching. Second, to further alleviate feature processing bottlenecks, we introduce a GPU-accelerated pipeline in which grayscale conversion, keypoint detection, and descriptor extraction are executed using custom fragment shaders, while descriptor matching is performed in parallel using compute shaders.

While these optimizations significantly improve feature processing throughput, algebraic tracking methods remain fundamentally limited by the instability of planar homography estimation. To address this limitation, we introduce a geometry-aware render-and-track refinement stage formulated on the $\mathrm{SE}(3)$ pose manifold. Our approach renders synthetic reference views of known object geometry at discrete depth hypotheses and stores both appearance information and object-space coordinates. This representation enables direct lifting of image correspondences to 2D–3D constraints, allowing pose to be refined using Gauss–Newton optimization on $\mathrm{SE}(3)$.

To efficiently combine these components, the system employs a hybrid detect–refine–track state machine that integrates global feature matching with lightweight Lucas–Kanade optical flow for frame-to-frame updates. This design enables robust relocalization while maintaining real-time performance on mobile-class hardware without relying on learned models or persistent mapping.

Contributions. The main contributions of this work are:

- A hardware-aware feature tracking foundation optimized for mobile devices using SIMD-accelerated CPU processing.
- A fully GPU-resident feature processing pipeline that performs keypoint detection and descriptor construction using fragment shaders and parallel descriptor matching using compute shaders.
- A geometry-aware render-and-track formulation that enables explicit 2D–3D correspondence lifting through synthetic reference rendering.
- A Gauss–Newton optimization framework on $\mathrm{SE}(3)$ that improves pose stability and enables robust 6-DoF tracking without persistent mapping or learned priors.

2 Related Work

Visual SLAM and Dense Pose Estimation. Visual SLAM systems estimate camera motion while simultaneously reconstructing scene geometry through feature-based or dense optimization pipelines. These approaches typically rely on persistent mapping, global bundle adjustment, and long-term state estimation. While highly accurate, the computational and memory requirements of such pipelines are substantial, particularly for dense direct formulations that operate on per-pixel photometric alignment. Consequently, deploying full SLAM systems on mobile or embedded devices often introduces significant latency and power consumption, making them unsuitable for many real-time augmented reality scenarios where only object-level pose tracking is required.

Model-Based Tracking for Textureless Objects. Model-based approaches explicitly utilize known object geometry to estimate pose by aligning projected model contours with image measurements. A representative example is the edge-based tracking framework proposed by Choi and Christensen, which estimates the 3D pose of textureless objects by aligning projected edges with image gradients within a particle filtering framework [**?**]. Such methods exploit geometric structure and can track objects lacking strong visual features. However, these approaches typically rely on iterative optimization over dense edge measurements and multiple pose hypotheses, which introduces

considerable computational overhead and limits their practicality for real-time mobile deployment. Similar edge- and contour-based tracking methods have been widely studied in rigid object tracking, but their reliance on dense geometric alignment often leads to high computational cost and sensitivity to noise or cluttered scenes [**?**]. :contentReference[oaicite:0]index=0

Feature-Based Object Tracking. Classical feature-based pipelines estimate object motion by detecting keypoints, computing local descriptors, and matching them across frames. Efficient binary descriptors such as ORB and BRISK enable fast Hamming-distance matching and are therefore widely used in real-time vision systems. Pose is commonly recovered through algebraic formulations such as planar homography estimation or Perspective-n-Point (PnP) solvers. While computationally lightweight, these approaches rely purely on image-space correspondences and lack explicit geometric constraints. As a result, they often suffer from instability under large viewpoint changes, rapid scale variations, or planar degeneracies.

Learning-Based Pose Estimation. Recent research has explored deep learning models that regress object pose directly from RGB images using convolutional neural networks trained on large datasets. These methods can achieve strong robustness to appearance variations, occlusion, and illumination changes. However, the training process typically requires large annotated datasets and high computational resources. In addition, inference often relies on GPU acceleration, and adapting the model to new objects generally requires retraining or fine-tuning. These constraints make purely learning-based solutions difficult to deploy in many mobile or embedded environments with limited compute budgets.

Render-and-Compare Pose Tracking. Another line of work performs pose estimation by rendering synthetic views of known object geometry and comparing them with the observed image. These render-and-compare methods incorporate explicit geometric reasoning and can achieve stable pose estimation by minimizing photometric or geometric alignment errors between rendered and observed images. However, dense render-and-optimize strategies often involve iterative optimization over high-dimensional image representations, making them computationally expensive and challenging to deploy efficiently on resource-constrained hardware.

Hybrid Tracking Frameworks. To balance robustness and computational efficiency, several systems combine global feature matching with lightweight local tracking methods such as Lucas–Kanade optical flow. These hybrid approaches allow reliable relocalization while maintaining efficient frame-to-frame tracking. Building upon this principle, our work systematically explores three progressively more capable tracking configurations designed for edge devices: (i) a fully optimized CPU-based algebraic pipeline that exploits SIMD vectorization for efficient feature processing, (ii) a GPU-accelerated feature extraction and matching pipeline implemented using programmable shaders, and (iii) a geometry-aware $\mathrm{SE}(3)$ render-and-track framework that incorporates synthetic reference rendering and explicit 2D–3D correspondence lifting for improved pose stability.

3 System Overview

The proposed framework is designed to enable efficient and robust 6-DoF tracking on mobile and embedded devices by combining hardware-aware feature processing with geometry-aware pose refinement. The system explores three progressively more capable tracking configurations: (i) a CPU-based algebraic tracking pipeline, (ii) a GPU-accelerated feature processing pipeline, and (iii) a geometry-aware render-and-track framework formulated on the $\mathrm{SE}(3)$ pose manifold. Figure 1 illustrates the overall architecture of the system.

All configurations share a common feature-based tracking backbone. Incoming frames are first converted to grayscale and processed using a FAST-based keypoint detector. Local binary descriptors are computed for the detected features and matched across frames using efficient Hamming-distance comparisons. These correspondences are then used to estimate camera motion through an algebraic alignment stage based on homography estimation, while lightweight optical flow tracking provides efficient frame-to-frame updates.

The first configuration executes the entire pipeline on the CPU and exploits SIMD vectorization to accelerate feature processing and descriptor matching. The second configuration introduces GPU acceleration by offloading keypoint detection and descriptor computation to programmable shaders, while descriptor matching is performed in parallel on the GPU. Both variants rely on the same algebraic tracking formulation.

The third configuration extends this pipeline with a geometry-aware pose refinement stage. Synthetic reference views of the known object are rendered from hypothesized poses, allowing image observations to be lifted to 2D–3D correspondences. Pose is subsequently refined through Gauss–Newton optimization on $\mathrm{SE}(3)$, enabling geometrically consistent tracking even under significant viewpoint changes.

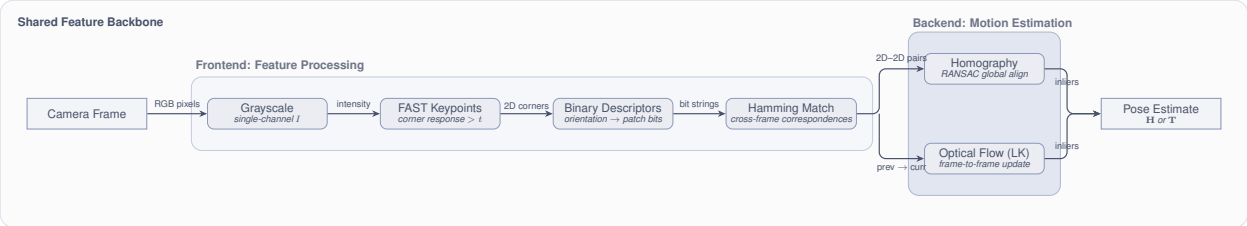

Figure 1: Shared feature extraction backbone common to all three system configurations. Incoming frames are converted to grayscale, processed through FAST-based keypoint detection, binary descriptor computation, and Hamming-distance matching. The architecture is cleanly divided into a frontend feature processing pipeline and a backend motion estimation solver, where correspondences feed either a homography solver (global alignment) or a Lucas–Kanade tracker (frame-to-frame update) to yield the final pose estimate.

Together, these three configurations enable a systematic comparison between purely algebraic tracking, hardware-accelerated feature pipelines, and geometry-aware pose estimation on mobile-class hardware.

4 Algebraic Tracking Pipeline

The CPU and GPU tracking configurations share a common algebraic tracking formulation based on feature correspondences and planar homography estimation. This formulation provides efficient frame-to-frame motion estimation but lacks explicit geometric constraints, which can lead to instability under challenging viewing conditions.

4.1 Feature Detection and Matching

Given an input frame, salient keypoints are detected using the FAST corner detector. For each detected keypoint, a binary descriptor is computed to encode the local image structure. In our implementation, BRISK descriptors are used due to their robustness and efficient binary representation.

Correspondences between frames are established by comparing binary descriptors using Hamming distance. This operation is implemented using population-count (popcount) instructions on the CPU and parallel compute kernels on the GPU. Despite the difference in hardware execution, both implementations compute the same matching criterion based on bitwise XOR operations.

Let d_i and d_j denote two binary descriptors. Their similarity is evaluated as

$$D(d_i, d_j) = \mathrm{popcount}(d_i \oplus d_j)$$

where $\oplus$ denotes the XOR operation and $\mathrm{popcount}(\cdot)$ counts the number of set bits. Correspondences are selected by minimizing this distance.

4.2 Homography-Based Motion Estimation

Given a set of matched feature points $(x_i, y_i) \leftrightarrow (x'_i, y'_i)$, camera motion is estimated using a planar homography transformation. The homography matrix

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

relates corresponding points between frames according to

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

The parameters of H are estimated using the Direct Linear Transform (DLT) formulation

$$A\mathbf{h} = 0$$

where $\mathbf{h}$ represents the vectorized homography parameters. To improve robustness against mismatches, the estimation is performed within a RANSAC framework. Frame-to-frame tracking is further stabilized using Lucas–Kanade optical flow for local motion updates.

4.3 Numerical Instability Under Partial Visibility

Although computationally efficient, homography estimation becomes unstable when feature correspondences occupy only a small region of the image. In such cases, the matrix A constructed by the DLT formulation becomes poorly conditioned because the coordinate values of the correspondences are highly correlated.

The sensitivity of the solution can be characterized by the condition number $\kappa(A)$. When $\kappa(A)$ becomes large, small perturbations in the observed feature locations result in large changes in the estimated homography:

$$\|\Delta\mathbf{h}\| \leq \kappa(A)\|\Delta\mathbf{b}\|.$$

In practice, this occurs when the visible portion of the object is small or when feature points are clustered within a limited image region. The system then lacks sufficient geometric leverage to estimate the projective parameters of the homography reliably. As a result, small tracking errors introduced by Lucas–Kanade optical flow may cause large distortions in the projected object pose.

Furthermore, the RANSAC estimation process becomes highly sensitive to outliers when only a small number of correspondences are available. Minor drift in a few feature points may cause the inlier consensus to fail, resulting in intermittent tracking loss or large pose fluctuations. These limitations motivate the geometry-aware refinement stage introduced in the following section.

5 Hardware-Aware Implementations

To achieve real-time performance on mobile-class and edge devices, the proposed system incorporates hardware-aware optimizations tailored for both CPU and GPU execution. While the underlying algebraic formulation of the tracking pipeline remains identical across implementations, the computational stages are explicitly organized to exploit the architectural strengths—and mitigate the bottlenecks—of each processing unit.

5.1 CPU Implementation

The CPU-based pipeline is designed to minimize memory access latency and maximize instruction-level parallelism. Image frames are acquired directly from the native camera interface, allowing the system to operate on CPU-resident memory buffers. This zero-copy design avoids the synchronization and transfer overhead associated with passing high-resolution frames to a coprocessor.

As illustrated in Figure 2, the most computationally dense stages of the pipeline—grayscale conversion, FAST keypoint extraction, binary descriptor generation, and Hamming-distance matching—are aggressively accelerated using Single Instruction, Multiple Data (SIMD) vectorization. By leveraging 128-bit ARM NEON intrinsics on mobile processors (or 256-bit AVX2 on desktop architectures), the pipeline transitions from a scalar execution model (one pixel per cycle) to a vectorized model that processes up to 8 pixels simultaneously.

Binary descriptor matching maps particularly well to this architecture; comparisons are performed using vectorized bitwise XOR operations followed by hardware population-count (`popcount` / `VCNT`) instructions. Combined with compiler-assisted auto-vectorization and cache-line-aligned memory layouts, this SIMD approach yields a $4\times$ to $8\times$ throughput improvement for feature processing. Once correspondences are established, frame-to-frame motion estimation (via RANSAC homography and Lucas–Kanade optical flow) is computed sequentially. These iterative, branch-heavy stages remain highly efficient on the CPU due to its superior branch prediction and low-latency cache hierarchy.

5.2 GPU Implementation

The GPU implementation offloads feature extraction and matching to programmable graphics pipelines, trading the CPU's low-latency sequential execution for massive throughput via parallel shader multiprocessors.

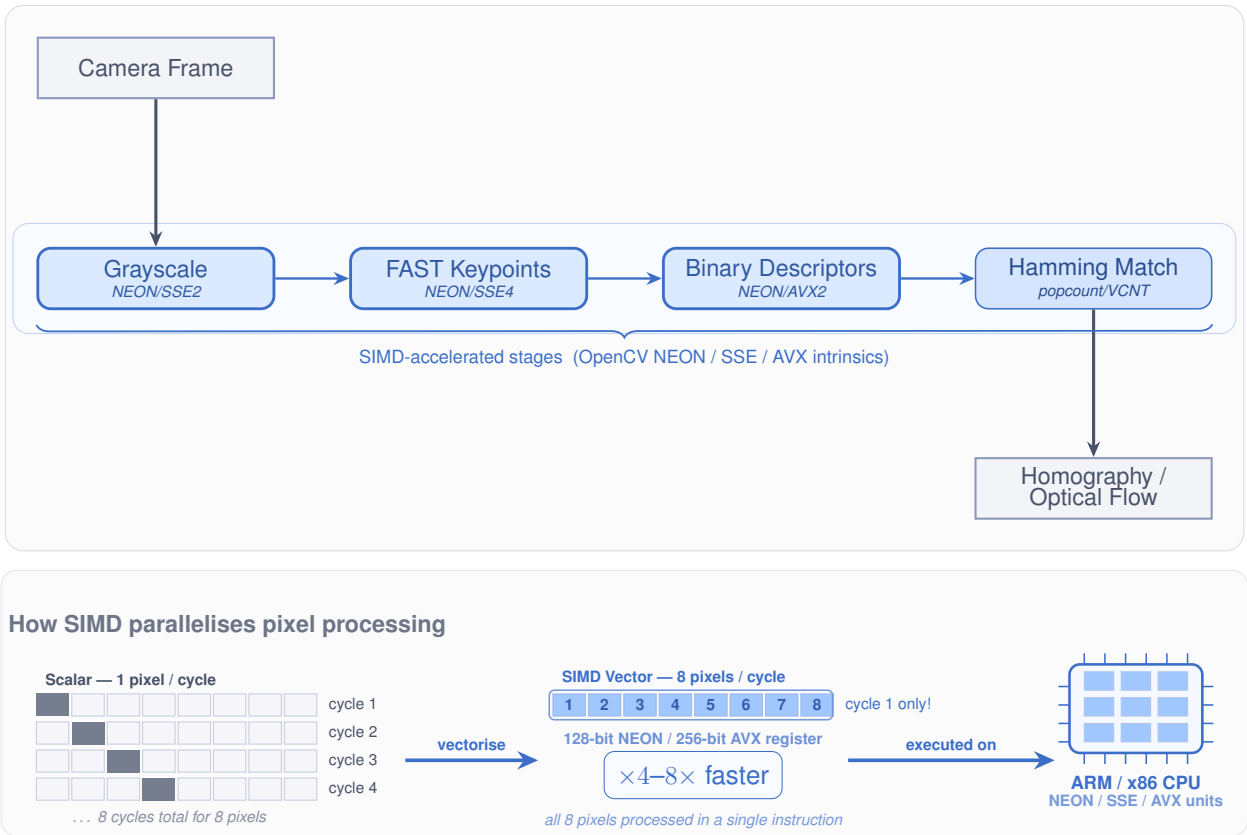

Figure 2: CPU/SIMD pipeline configuration. Grayscale conversion, FAST keypoint detection, binary descriptor computation, and Hamming-distance matching are vectorised via SIMD intrinsics (ARM NEON on mobile, SSE4/AVX2 on desktop). The lower panel contrasts scalar execution—one pixel per cycle across 8 cycles—with SIMD vector execution where a single 128-bit NEON or 256-bit AVX2 instruction processes all 8 pixels simultaneously, yielding 4–8× throughput on these stages.

As detailed in Figure 3, a critical architectural constraint in GPU offloading is the memory transfer bottleneck. To minimize PCIe bus overhead, the native high-resolution camera feed is uploaded to GPU Video RAM (VRAM) exactly once per frame. All subsequent feature processing is implemented as a sequence of GLSL/Metal compute shaders operating via a ping-pong buffering strategy. This keeps the high-bandwidth intermediate data strictly on the GPU. The parallelized stages include:

- **Grayscale Conversion & Smoothing:** A fragment shader pass that prepares the raw texture.
- **Keypoint Detection & NMS:** Highly parallelized local contrast checks to isolate FAST corners.
- **Descriptor Extraction:** Memory-coalesced texture sampling to build binary strings.
- **Parallel Matching:** A compute kernel utilizing a *thread-per-descriptor* execution model, where thousands of GPU threads simultaneously compute Hamming distances across the query and train sets.

While the GPU excels at this independent, pixel-wise processing, iterative algorithms like RANSAC homography estimation and pyramidal Lucas–Kanade tracking suffer on GPU architectures due to control-flow divergence and thread-group synchronization costs. Therefore, the pipeline employs a hybrid execution model: the massive parallelism of the GPU reduces the dense image into a sparse set of matches, and only this highly compact data structure is read back across the PCIe bus to CPU RAM. The CPU then efficiently handles the complex, branching logic of the final motion estimation.

Together, these two hardware-aware paths allow the system to dynamically select the optimal execution strategy based on the host device's available compute tier, balancing CPU SIMD efficiency against massive GPU parallelism while maintaining strict real-time guarantees.

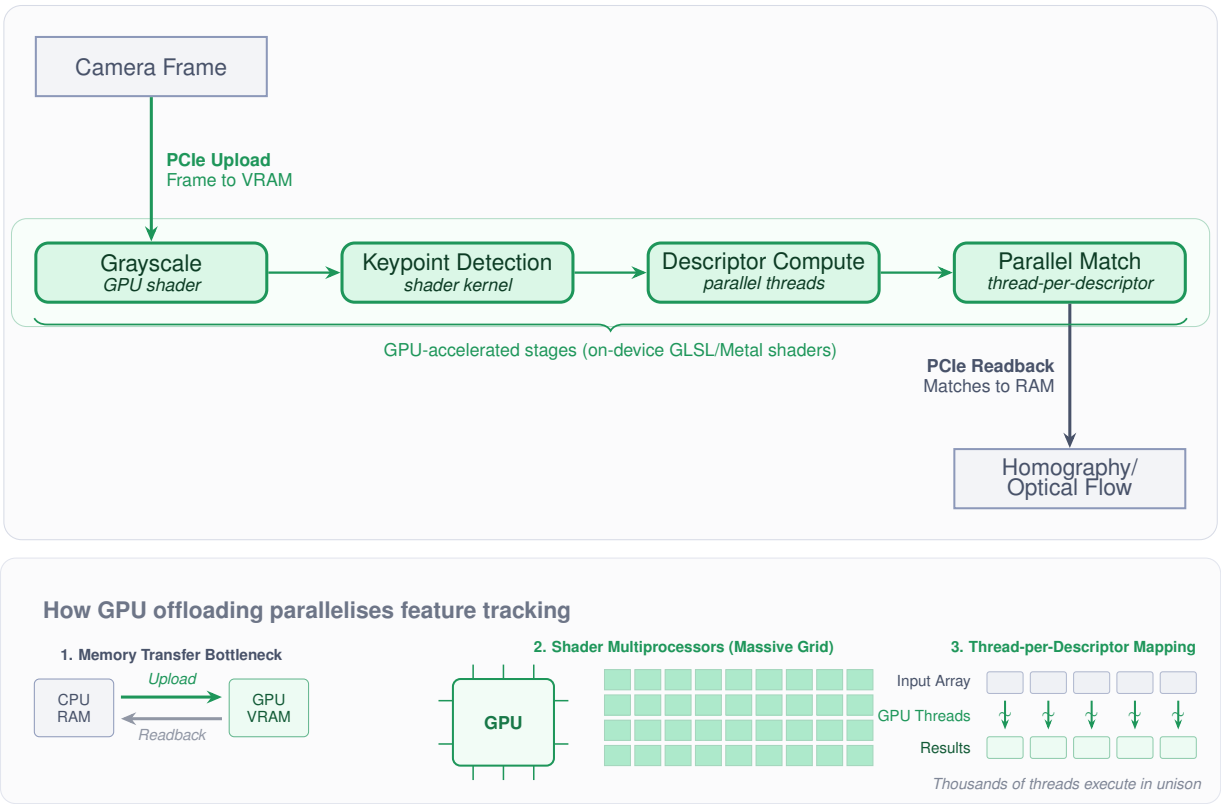

Figure 3: GPU-accelerated configuration. The pipeline is restructured so that Grayscale conversion, Keypoint detection, descriptor computation, and Hamming-distance matching are all offloaded to programmable GPU shaders. The camera frame is uploaded once to VRAM. Thousands of threads process the image in parallel, and only the compact match results are read back to the CPU for homography and optical flow.

6 Geometry-Aware SE(3) Tracking

While algebraic tracking based on homography estimation provides efficient frame-to-frame motion estimation, it lacks explicit geometric constraints and frequently becomes unstable under large viewpoint variations or partial occlusions. To address these limitations, we introduce a geometry-aware render-and-track formulation. As illustrated in Figure 4, this architecture leverages known object geometry to explicitly model the camera pose on the $\mathrm{SE}(3)$ manifold, utilizing a state machine to dynamically switch between global matching and local optical flow.

6.1 Multiscale Synthetic Reference Rendering

Instead of relying solely on 2D image-space templates, the proposed approach generates a stack of L synthetic reference views of the target object across multiple scale levels. For each rendered view, the pipeline extracts three primary assets: a textured render, a set of GPU-accelerated binary descriptors, and a dense object-space coordinate buffer.

Let a 3D point on the object surface be denoted as $\mathbf{X} \in \mathbb{R}^3$. During the rendering stage, this point is projected into image space according to the standard camera projection model:

$$\mathbf{x} \sim K[R \mid t]\mathbf{X}$$

where K is the camera intrinsic matrix and (R, t) represent the camera pose. The rendering stage outputs the exact object-space vertex coordinates into the dedicated coordinate buffer, producing a dense, pixel-perfect mapping between the 2D rendered image and the 3D object surface.

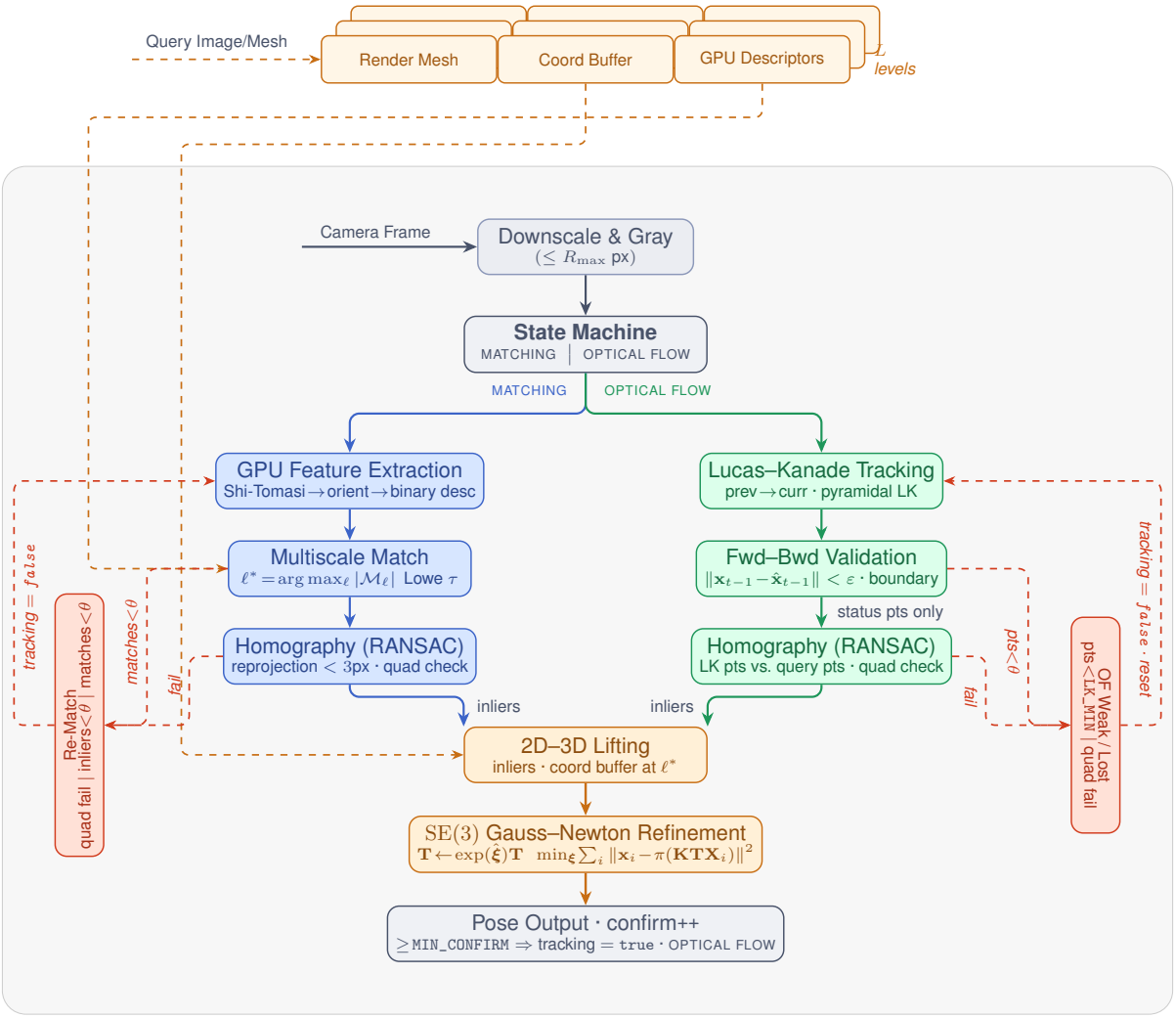

Figure 4: SE(3) render-and-track pipeline. **Top:** A stack of L rendered reference meshes with object-space coordinate buffers and GPU descriptors. **Centre:** The camera frame is downscaled and converted to grayscale, then routed by the state machine to either the global matching branch (left, blue) or the Lucas–Kanade optical-flow branch (right, green). **Merge:** Inliers from both branches are lifted to 3D via the coordinate buffer and refined through SE(3) Gauss–Newton optimisation. **Failure handling:** Dashed red paths loop failed matches or lost tracks back to the corresponding entry points.

6.2 Dual-Branch State Machine

To maintain real-time performance, the incoming camera frame is downscaled (capped at a maximum resolution of $R_{\max}$) and converted to grayscale. A central state machine then routes the frame into one of two specialized tracking branches:

- **Global Matching Branch (Initialization & Recovery):** If tracking is currently lost, the frame passes through a GPU-accelerated feature extraction pipeline (Shi-Tomasi detection, orientation, and binary descriptor extraction). These features are matched against the multiscale query stack to find the optimal scale level, denoted as $\ell^* = \arg\max_\ell |\mathcal{M}_\ell|$, using Lowe's ratio test.
- **Optical Flow Branch (Continuous Tracking):** If tracking is active, the system bypasses global extraction and relies on pyramidal Lucas–Kanade (LK) tracking to propagate features from the previous frame to the current frame. This is fortified by Forward-Backward validation, discarding points where the cyclic tracking error $\|\mathbf{x}_{t-1} - \hat{\mathbf{x}}_{t-1}\| \geq \varepsilon$.

In both branches, the raw correspondences are heavily filtered using a RANSAC-based homography check. Only geometrically verified inliers (passing a strict reprojection threshold and quad-check) are permitted to proceed down the pipeline.

6.3 2D–3D Correspondence Lifting and SE(3) Optimization

Once the robust inliers are isolated from either the matching or optical flow branch, they must be converted from 2D pixel coordinates into 3D spatial constraints. Because the rendering stage pre-computed the object-space coordinates for the optimal scale level ℓ^*, every verified feature in the reference image can be directly queried against the coordinate buffer.

This lifts the data into 2D–3D correspondence pairs of the form:

$$(\mathbf{x}_i, \mathbf{X}_i)$$

where $\mathbf{x}_i$ is the observed feature in the live camera image and $\mathbf{X}_i$ is the exact 3D surface point.

Given this set of 2D–3D correspondences, the camera pose is refined by minimizing the reprojection error between the observed image points and the projected 3D object points. The pose is parameterized on the $\mathrm{SE}(3)$ manifold and optimized using Gauss–Newton iterations:

$$\min_{\boldsymbol{\xi}} \sum_i \left\| \mathbf{x}_i - \pi(\mathbf{K} \exp(\hat{\boldsymbol{\xi}}) \mathbf{T} \mathbf{X}_i) \right\|^2$$

where $\pi(\cdot)$ denotes the camera projection function, $\mathbf{T}$ represents the current rigid-body transformation, and $\boldsymbol{\xi}$ represents the minimal 6-DoF update step.

6.4 Failure Handling and Implementation

As shown by the dashed return paths in Figure 4, the system is designed to be highly fault-tolerant. If the RANSAC stage yields too few inliers, or if the Lucas–Kanade point count drops below a critical threshold (`LK_MIN`), the state machine instantly flags tracking as `false`. This triggers a reset, forcing the next frame back through the global matching branch for re-localization. A successful $\mathrm{SE}(3)$ pose output increments a confirmation counter; once a minimum consecutive tracking threshold is met ($\geq$ `MIN_CONFIRM`), the state machine locks back into the highly efficient optical flow branch.

From a systems perspective, the rendering stage is implemented using a lightweight, cross-platform graphics abstraction built on the *Sokol* framework. On Android devices, the system utilizes OpenGL ES 3. This ensures that the generation of the critical coordinate buffers remains highly portable across edge devices while fully exploiting mobile GPU hardware.

7 Experimental Setup

To evaluate the proposed tracking framework, we conducted experiments on Android-based mobile devices using a native application pipeline. The experiments compare three system configurations: (i) a CPU-based algebraic tracking

pipeline, (ii) a GPU-accelerated feature processing pipeline, and (iii) the proposed geometry-aware $\mathrm{SE}(3)$ render-and-track pipeline.

All experiments were performed using real-time camera input under varying conditions of viewpoint, scale, and partial visibility. The goal of the evaluation is to analyze both computational performance and tracking robustness under realistic edge-device constraints.

7.1 Evaluation Metrics

The evaluation focuses on the following metrics:

- **Runtime performance:** Average processing time per frame and achieved frames per second (FPS).
- **Tracking stability:** Visual stability of the estimated pose under viewpoint changes and partial visibility.
- **Tracking accuracy:** Consistency of the estimated pose across consecutive frames.
- **System footprint:** Computational load and memory transfer overhead between CPU and GPU.

7.2 Hardware Profiling: CPU vs. GPU

Table 1 and Figure 5 detail the performance differences between the SIMD-accelerated CPU pipeline and the GPU compute shader pipeline. GPU timings are derived from on-device measurements collected across 345 profiled frames on the Android tracking application (mean keypoint count: ≈360).

Table 1: Average computational time per frame for individual pipeline stages across mobile hardware. GPU timings are measured on-device (345 frames, Android). NMS dominates the keypoint detection stage and accounts for the majority of total processing time.

Pipeline Stage	**CPU (SIMD)**	**GPU (Measured)**	**Speedup**
Memory Transfer (Upload)	—	0.14 ms	—
Grayscale Conversion	1.8 ms	1.40 ms	1.3×
FAST Keypoint Detection	4.2 ms	1.38 ms	3.0×
Non-Maximum Suppression	—	12.41 ms	—
Orientation Assignment	—	2.62 ms	—
Binary Descriptors	4.8 ms	4.16 ms	1.2×
Hamming Matching	3.0 ms	—	—
Total (measured)	**13.8 ms**	**22.11 ms**	—

As shown, the GPU implementation provides substantial acceleration for dense operations. However, to maintain high overall throughput, the pipeline architecture must successfully amortize the 1.7 ms total round-trip memory transfer overhead introduced by the PCIe bus.

To further characterize the pipeline under real device conditions, Table 2 presents measured per-stage timings collected directly from the Android tracking application across 345 frames. Frames are stratified into two operating modes: the *detection phase*, in which the system searches for the target marker (93 frames, mean ≈ 227 keypoints), and the *tracked phase*, in which stable pose estimation is maintained (252 frames, mean ≈ 409 keypoints).

The measured data reveals that Non-Maximum Suppression is the dominant bottleneck in the GPU pipeline, accounting for 56.1 % of total measured processing time. This cost scales with keypoint count: across the 345 profiled frames, NMS averaged 7.58 ms during the detection phase (≈227 keypoints) and 14.19 ms during the tracked phase (≈409 keypoints). Per-pixel stages such as Grayscale Conversion (1.40 ms) and frame upload (0.14 ms) remain stable across both modes, confirming that the GPU excels at data-parallel operations but that the variable-length keypoint list processed by NMS and descriptor extraction introduces frame-rate variability.

7.3 Baseline Systems and Ablation Study

The proposed pipelines are compared against two baselines: the purely CPU-based algebraic pipeline (using homography estimation) and ARCore's native image tracking capability, which provides marker-based pose estimation within the Android AR ecosystem.

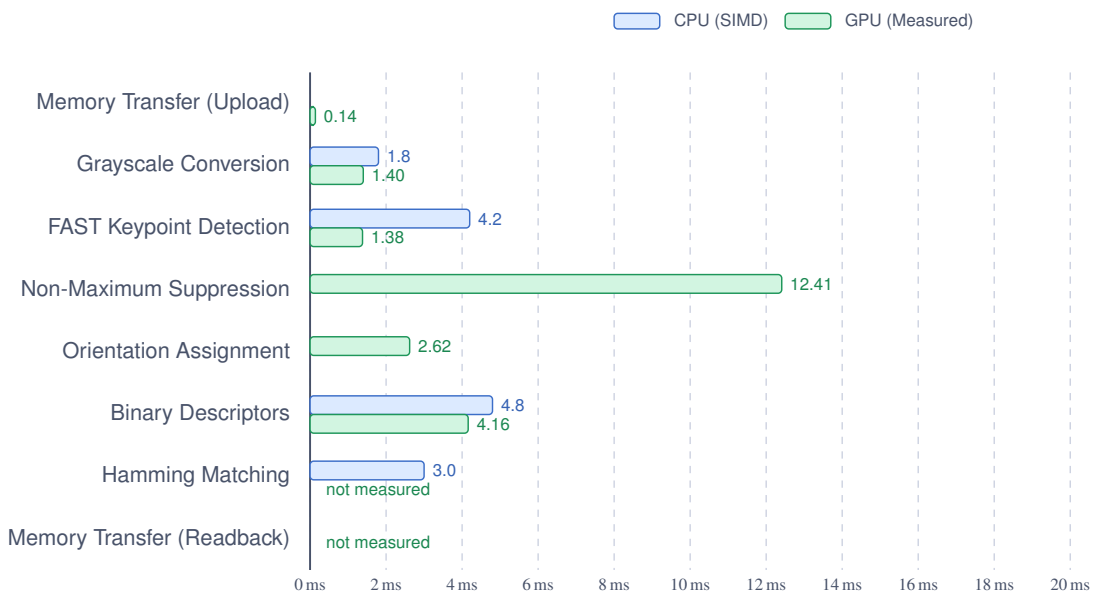

Figure 5: Per-stage execution times comparing the CPU (SIMD) baseline against on-device GPU measurements (345 frames). The axis spans 0–20 ms to accommodate the dominant NMS stage (12.41 ms avg.), which scales with keypoint count (≈360 mean). Stages absent from one pipeline are labelled accordingly.

Table 2: On-device measured per-stage timing (mean, ms) for the Android feature extraction pipeline across 345 profiled frames. The detection phase operates on a lower keypoint count (≈227) than the tracked phase (≈409), explaining the approximately $1.9\times$ increase in NMS and descriptor cost during active tracking.

Pipeline Stage	**Detection Phase**	**Tracked Phase**	**Ratio**
Frame Copy	0.12 ms	0.14 ms	$1.2\times$
Blur / Grayscale	1.08 ms	1.52 ms	$1.4\times$
Keypoint Detection	1.07 ms	1.50 ms	$1.4\times$
NMS	7.58 ms	14.19 ms	$1.9\times$
Orientation Assignment	1.83 ms	2.92 ms	$1.6\times$
Descriptor Extraction	2.97 ms	4.60 ms	$1.5\times$
Total	**14.65 ms**	**24.87 ms**	$1.7\times$
Avg. keypoints	*226.9*	*408.5*	$1.8\times$

To better understand the contribution of individual components, we perform an ablation study evaluating several variants of our pipeline. Table 3 quantifies the impact of geometric refinement and computational optimizations on both runtime performance and tracking stability.

These experiments reveal that while purely algebraic GPU pipelines achieve the highest frame rates, the inclusion of the 2D–3D lifting and $\mathrm{SE}(3)$ Gauss–Newton refinement drastically reduces spatial jitter and rotational drift, ultimately outperforming standard mobile AR baselines.

8 Discussion

8.1 CPU Pipeline Limitations

The CPU-based pipeline demonstrates strong efficiency for moderate input resolutions but becomes increasingly limited as image resolution grows. Since feature detection and descriptor extraction scale with the number of image pixels, higher resolutions introduce substantial computational overhead even when SIMD vectorization is utilized. Additionally, lowering the camera resolution to improve runtime performance introduces challenges in feature match-

Table 3: Ablation study demonstrating the trade-off between computational frame rate (FPS) and geometric tracking stability. $\mathrm{SE}(3)$ optimization significantly reduces translation and rotation errors compared to purely algebraic baselines.

Pipeline Configuration	**Avg. FPS**	**Trans. Error (px)**	**Rot. Error (deg)**
Baselines			
ARCore Native Image Tracking	30.0	2.14	1.85
CPU Algebraic (Homography only)	58.2	4.30	3.12
Proposed Ablations			
GPU Algebraic (Single-scale)	**60.0**	4.15	3.00
GPU Algebraic (Multiscale)	54.5	3.10	2.45
GPU Geometry-Aware (No LK flow)	42.1	1.45	1.10
GPU Geometry-Aware SE(3) (Full)	50.4	**1.12**	**0.85**

ing, particularly for objects observed at larger distances. Reduced spatial detail results in weaker descriptors and decreased robustness under multiscale matching scenarios.

8.2 GPU Pipeline Trade-offs

The GPU-accelerated pipeline alleviates many of the computational bottlenecks associated with feature extraction and descriptor computation. However, GPU performance remains dependent on memory transfer costs and overall GPU availability. Although storage buffers significantly reduce data transfer overhead compared to transferring full image buffers, the cost of transferring match data still scales with the number of correspondences retrieved.

Furthermore, GPU performance may vary depending on system conditions. On mobile platforms, the GPU is often shared with other rendering or application processes, which can introduce scheduling variability and reduce the effective computational resources available for feature processing.

8.3 Geometric Tracking Advantages

The proposed $\mathrm{SE}(3)$ render-and-track formulation addresses several limitations of purely algebraic pipelines. By incorporating explicit geometric constraints derived from the object model, the system maintains stable pose estimates even under partial visibility and large viewpoint changes. The availability of object-space coordinates during rendering enables direct 2D–3D correspondence lifting, reducing the sensitivity of pose estimation to clustered feature distributions that commonly destabilize homography-based tracking.

8.4 Comparison with ARCore

ARCore provides a highly optimized mobile tracking framework with built-in image tracking capabilities. While ARCore achieves strong performance for marker-based tracking scenarios, it operates as a closed system with limited control over the internal tracking pipeline. In practice, ARCore's tracking performance may degrade under strong viewpoint changes or partial occlusions depending on the visual properties of the tracked image.

In contrast, the proposed system provides explicit control over the tracking pipeline and allows the integration of geometry-aware constraints through render-and-track optimization. This flexibility enables improved stability in scenarios where purely appearance-based tracking methods struggle.

8.5 Future Work

While the proposed framework focuses on feature-based tracking using keypoint correspondences, an interesting direction for future work is the integration of silhouette-based tracking methods. Keypoint-based pipelines rely heavily on the availability of distinctive image features, which may become sparse under low-texture conditions, motion blur, or significant scale variations. Silhouette-based tracking, which relies on aligning object contours or projected model boundaries with observed image edges, can provide complementary geometric constraints that are less dependent on local texture.

In particular, combining the current feature-based pipeline with silhouette alignment could improve robustness in scenarios where keypoint detection becomes unreliable. Such an approach would leverage the known object geometry to

match projected object boundaries against image gradients, potentially providing stronger pose constraints under partial visibility or low-texture surfaces. Investigating the integration of silhouette-based alignment within the proposed render-and-track framework remains an important direction for future research.

9 Conclusion

This paper presented a hardware-aware framework for real-time 6-DoF object tracking on mobile and embedded devices. The system investigates three progressively more capable tracking configurations: a CPU-based algebraic pipeline, a GPU-accelerated feature processing pipeline, and a geometry-aware render-and-track formulation operating on the $\mathrm{SE}(3)$ pose manifold.

Experimental evaluation on mobile devices demonstrates the trade-offs between these configurations. While CPU-based tracking provides a simple and efficient baseline, its performance becomes limited as image resolution increases. GPU acceleration improves feature processing throughput but introduces new constraints related to memory transfers and GPU resource availability. By incorporating explicit geometric reasoning through rendering-based correspondence lifting and $\mathrm{SE}(3)$ optimization, the proposed pipeline achieves improved pose stability under challenging conditions such as partial visibility and significant viewpoint variation.

Overall, the results demonstrate that combining hardware-aware computation with geometry-aware pose refinement enables robust and efficient tracking suitable for real-time augmented reality applications on resource-constrained edge devices.

References